

Attribution/License

- Original Materials developed by Mike Shah, Ph.D. (www.mshah.io)
- This slideset and associated source code may not be distributed without prior written notice
- This slideset may not be mined using AI or algorithms and otherwise generating derivative products without permission.

Please do not redistribute slides/source without prior written permission.



3D Graphics Engine in D

with Mike Shah

```
1 void main()  
2 {  
3     import std.algorithm, std.stdio;  
4  
5     "Starting program".writeln;  
6  
7     enum a = [ 3, 1, 2, 4, 10 ];  
8     // Sort data at compile-time  
9     static immutable b = sort(a);  
10
```

Web: mshah.io
YouTube: www.youtube.com/c/MikeShah
Social: mikeshah.bsky.social
Courses: courses.mshah.io
Talks: <http://tinyurl.com/mike-talks>

35 minutes | Audience: All
2:45pm - 3:20pm Fri, Sept 4, 2026

DConf

Abstract (which you already read :))

Talk Abstract: The D programming language is well positioned to support game and graphics development because of its ability to compile fast, generate fast code, and maintain plasticity for refactoring code at scale. In this talk, Mike will demonstrate several graphics tech demos that he has developed in his graphics engine using OpenGL 4.6. Throughout the talk Mike will highlight what features of the D programming language helped his development alongside his graphics engine. Mike will also show off his dub configuration, and VIM setup to show why he believes the barrier to entry to using D for game development is low and provides an excellent point to get started building real projects with D!



Web

www.mshah.io



<https://www.youtube.com/c/MikeShah>

Non-Academic Courses

courses.mshah.io

Conference Talks

<http://tinyurl.com/mike-talks>

Your Tour Guide for Today

Mike Shah

- **Current Job:** Teaching Faculty at **Yale University**
 - **Teach/Research:** computer systems, graphics, geometry, game engines, software engineering, and computer science education.
- **Available for:**
 - **Technical Training/Talks:**
 - GDB Training, C++, D Language, or other speaking engagements
- **Fun:**
 - Enjoy travel, NBA basketball, Olympics, running/weights, video games, music, DuoLingo (French mostly), and pizza making



Web

www.mshah.io



YouTube

<https://www.youtube.com/c/MikeShah>

Non-Academic Courses

courses.mshah.io

Conference Talks

<http://tinyurl.com/mike-talks>

Find my programming content (C++,D, C, graphics) on YouTube

<https://www.youtube.com/c/mikeshah>



C3 - First Impression [Programming Languages Episode 31]
6.8K views • Streamed 1 year ago



Serialize and Deserialize a struct in C++ - Stream-Based I/O part 8 of n- Modern Cpp Series Ep. 198
3.8K views • 10 months ago



Cpp2 - First Impression [Programming Languages Episode 27]
4.3K views • 1 year ago



Delegates (and review on function lambda) [Dlang Episode 135]
201 views • 4 days ago

Popular videos



In 54 Minutes, Understand the whole C and C++...
76K views • 4 years ago



C++ Video Series Introduction | Modern Cpp...
68K views • 3 years ago



[Ep. 1] What is the Simple Directmedia Layer (SDL) an...
53K views • 3 years ago



Learn the lldb debugger basics in 11 minutes | 2021...
48K views • 3 years ago



[Setup Video] Setting up C++ on Mac (Shown on Apple M...
45K views • 3 years ago



The what and the why of concurrency | Introduction t...
38K views • 3 years ago

The C++ Programming Language



The C++ Programming Language
Public • Playlist
View full playlist



Modern C++ (cpp) Concurrency
Public • Playlist
View full playlist



C++ Software Design and Design Patterns
Public • Playlist
View full playlist



wxWidgets Graphical User Interface (GUI) Programmin...
Public • Playlist
View full playlist



C++ and Pybind11
Public • Playlist
View full playlist



C++ Quick Start
Public • Playlist
View full playlist



Web

www.mshah.io



YouTube

<https://www.youtube.com/c/MikeShah>

Non-Academic Courses

courses.mshah.io

Conference Talks

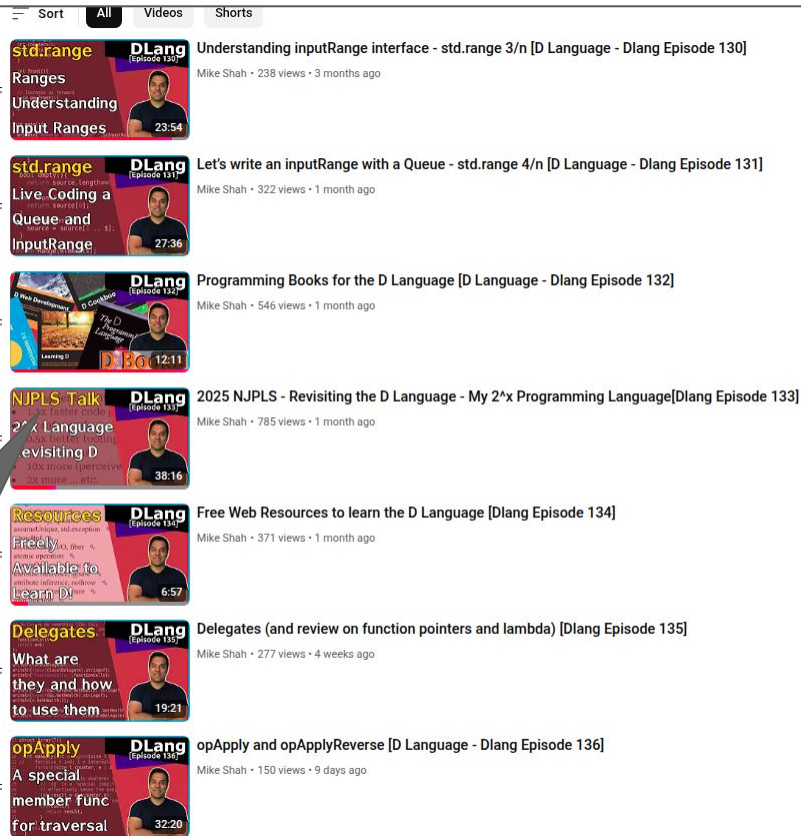
<http://tinyurl.com/mike-talks>

Find my programming content (C++,D, C, graphics) on YouTube

<https://www.youtube.com/c/mikeshah>



Search my
YouTube for
resources on
learning the D
language



Web

www.mshah.io



YouTube

<https://www.youtube.com/c/MikeShah>

Non-Academic Courses

courses.mshah.io

Conference Talks

<http://tinyurl.com/mike-talks>

Find my programming content (C++,D, C, graphics) on YouTube
<https://www.youtube.com/c/mikeshah>

A YouTube video player thumbnail. At the top, a dark blue banner contains the text "[Episode 0]" in white and "Series Teaser" in a larger white font. Below this, the main title "matrix.py matrix.d" is displayed in a large, white, monospaced font. To the right of the title is a circular profile picture of a man with glasses and a beard, wearing a dark t-shirt. Below the title, the text "D Lang" is written in a large, white, cursive font. A small white icon of a pencil inside a circle is located at the bottom right of the video frame. Below the video frame, the text "D Language (DLang) Programming" is written in a large, bold, black font. Underneath the title, it says "by Mike Shah" next to a small circular profile picture of the same man. Below that, it says "Playlist • Public • 140 videos • 46,632 views". A paragraph of text follows: "A full playlist on learning the D Programming language. A great starting place for beginners to start, as well as... more". At the bottom, there is a white button with a black play icon and the text "Play all". To the right of this button are four circular icons: a plus sign, a pencil, a share icon, and a vertical ellipsis.

Sort

All

Videos

Shorts

std.range

DLang

Episode 132

Understanding inputRange interface - std.range 3/n [D Language - Dlang Episode 130]

Mike Shah • 238 views • 3 months ago

std.range

Ranges

Understanding input Ranges

23:54

std.range

Live

Que

Input

std.range

Free Web Resources to learn the D Language [Dlang Episode 134]

Mike Shah • 371 views • 1 month ago

std.range

Resources

Episode 133

Free Web Resources to learn the D Language [Dlang Episode 134]

Mike Shah • 371 views • 1 month ago

std.range

Delegates

Episode 135

Delegates (and review on function pointers and lambda) [Dlang Episode 135]

Mike Shah • 277 views • 4 weeks ago

std.range

opApply

Episode 136

opApply and opApplyReverse [D Language - Dlang Episode 136]

Mike Shah • 150 views • 9 days ago

std.range

opApply

Episode 136

opApply and opApplyReverse [D Language - Dlang Episode 136]

Mike Shah • 150 views • 9 days ago

Okay -- enough about me

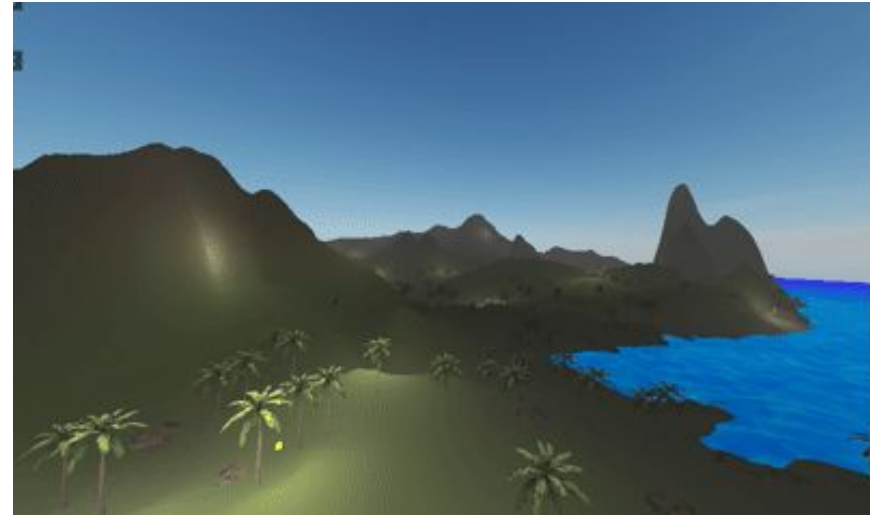
-- on to the talk!



*** Live Demo ***

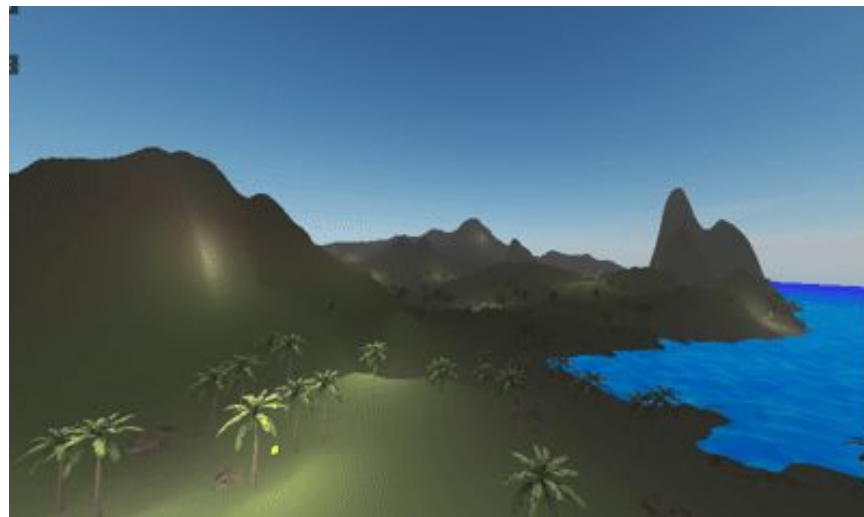
3D Graphics Engine Demo!

- Live demo checklist
 - Engine Side
 - Adding components
 - Automatic management of uniforms
 - Graphics effects
 - object loading
 - water
 - multi-textured terrain
 - instancing
 - picking
 - lights
 - fog



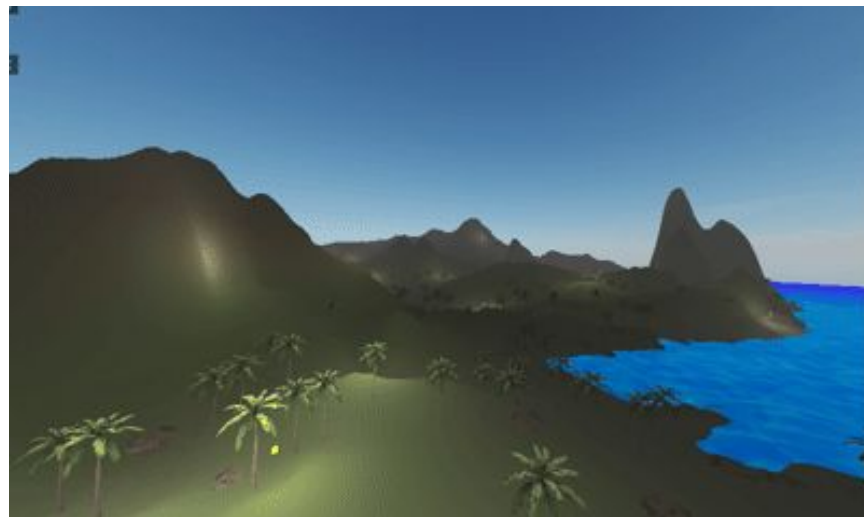
Goals of this Project (1/2)

- To write a reusable game and graphics engine 100% in D
 - Ultimately this supports the main goal of **shipping a game**.
 - Having a playground to experiment with engine and graphics code quickly is important, and why I am investing in D.



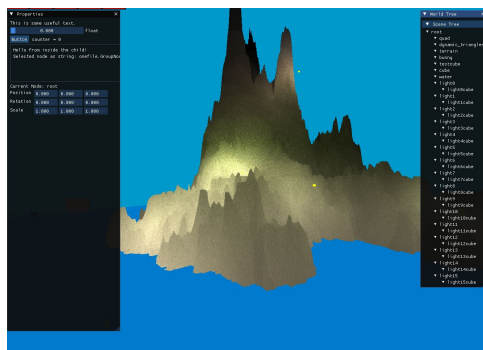
Goals of this Project (2/2)

- Engine should be simple to use
 - User either writes glsl code or D code -- engine takes care of integration.
- Cross-Platform
 - Desktop
 - Mobile Targets (Android)
 - Other devices
 - (e.g. steamdeck)
 - Minimize dependencies as much as possible (e.g. avoid libraries for loading assets, etc.) -- own the code.
- Modern 'enough' Rendering
 - I don't need to beat Unreal, but I should be able to write some cool effects with vertex, fragment, tessellation, and compute shaders at the least.

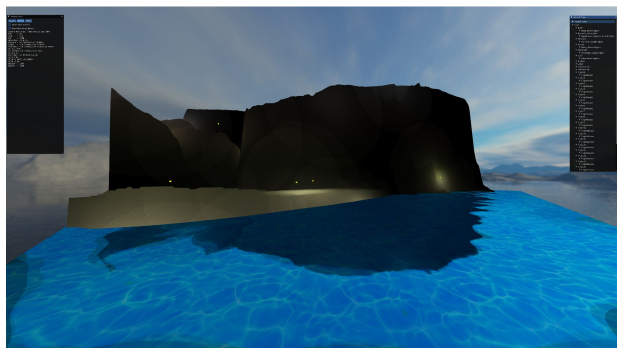


Today's talk

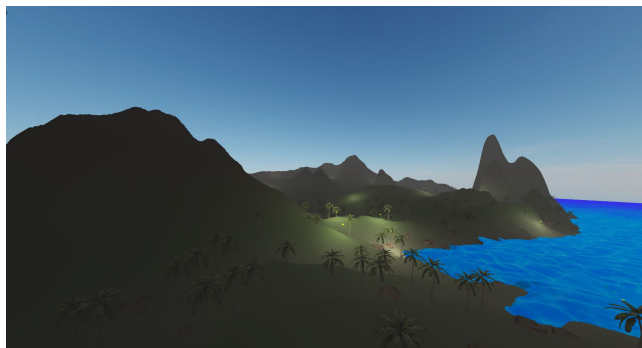
- I'll highlight how the **D Language** helped solve some interesting problems, and importantly helped me iterate very quickly!
 - (Fast enough I could take a little homework assignment and rapidly iterate and improve the results!)



1-2 weeks of progress



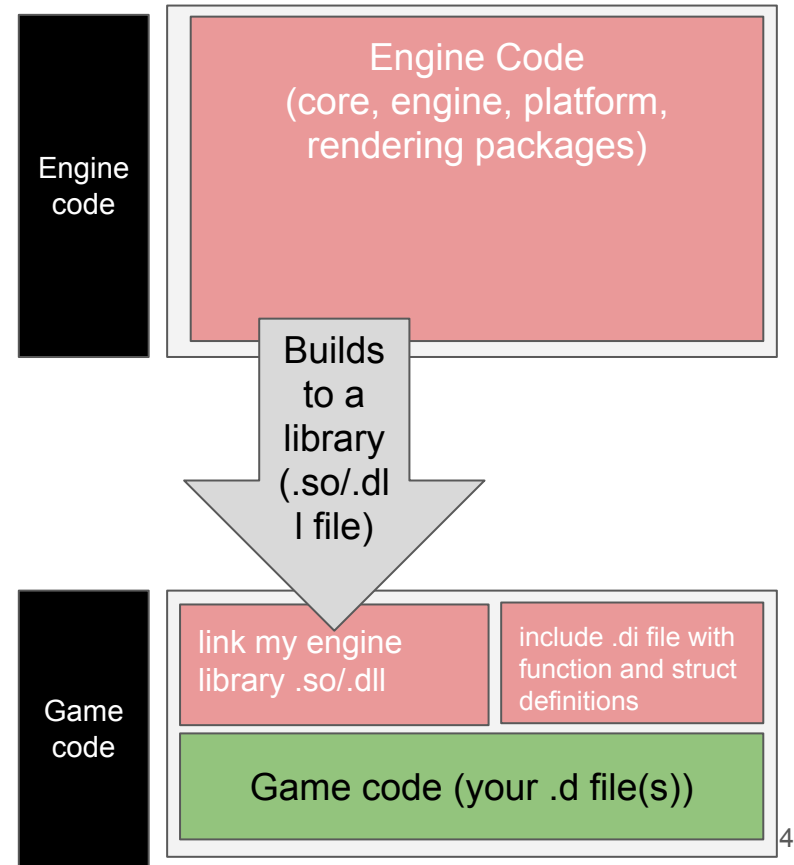
1 more week of progress



Engine Structure

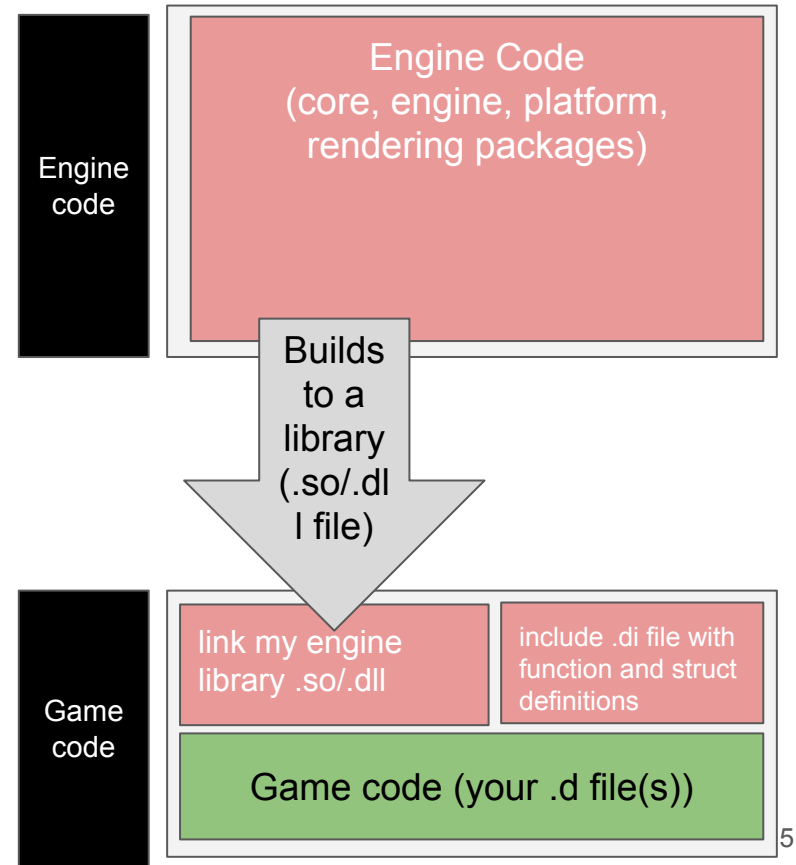
3D Graphics/Game Engine Structure (1/2)

- For a reusable engine, one of the first things I did was separate out engine from game code.
 - Engine Code is a shared library (.dll/.so/.dylib)
 - Game Code loads shared library and an interface file (.di)
 - (User writes their code here)



3D Graphics/Game Engine Structure (2/2)

- So -- how to distribute the engine code?
- Ultimately I'd like needs to be simple and have
 - **Game/Graphics engine -- one library file, one .di file**



Game/Graphics engine -- one library, one .di file: Attempt # 1

- I tried playing around with creating a 'public interface' directory for my project
 - **-H**, to generate the header interface
 - **-op**, to preserve the directory contents
 - **-Hd=publicinterface**, as the directory
- The issue however, is preserving this 'hierarchy' seemed to make things difficult to link -- this also is not 'one file' but 'many files'

```
publicinterface/  
├── core  
│   ├── application.di  
│   ├── error.di  
│   └── geometry  
│       ├── dynamicgeometry.di  
│       ├── objgeometry.di  
│       ├── package.di  
│       ├── surface.di  
│       ├── terraingeometry.di  
│       └── vertexformats.di
```

Game/Graphics engine -- one library, one .di file: Attempt # 2 (1/2)

- Then I tried something hacky and concatenated all the .di files together in different configurations

```
1989     uint mXDimensions;
1990     uint mZDimensions;
1991     float[][] mHeightValues;
1992     this(string surfaceName, uint xDim, uint zDim, string heightmap_file);
1993     override void Render();
1994     void MakeTerrain(uint xDim, uint zDim, string heightmap_file);
1995     float GetTerrainHeight(int xTile, int zTile);
1996 }
1997 // D import file generated from 'source/core/mesh.d'
1998 class MeshNode : ISceneNode
1999 {
2000     ISurface mSurface;
2001     IMaterial mMaterial;
2002     this(string name, ISurface surface, IMaterial material);
2003     override string toString();
2004     override void Update();
2005     IMaterial GetMaterial();
2006     ISurface GetSurface();
2007 }
2008 // D import file generated from 'source/engine/scene.d'
2009 class Scene
2010 {
2011     Application mApplicationRef;
2012     string mName;
2013     ResourceManager mResourceManager;
2014     SceneTree mSceneTree;
2015     SceneTreeFilterFunction mSceneTreeFilter_func = null;
2016     Camera mCamera;
```

Game/Graphics engine -- one library, one .di file: Attempt # 2 (2/2)

- Also a disaster, in the sense that I really struggled with how to import the symbols.
 - This would potentially make a user have to know some package/module structure

```
1989 uint mXDimensions;
1990 uint mZDimensions;
1991 float[][] mHeightValues;
1992 this(string surfaceName, uint xDim, uint zDim, string heightmap_file);
1993 override void Render();
1994 void MakeTerrain(uint xDim, uint zDim, string heightmap_file);
1995 float GetTerrainHeight(int xTile, int zTile);
1996 }
1997 // D import file generated from 'source/core/mesh.d'
1998 class MeshNode : ISceneNode
1999 {
2000     ISurface mSurface;
2001     IMaterial mMaterial;
2002     this(string name, ISurface surface, IMaterial material);
2003     override string toString();
2004     override void Update();
2005     IMaterial GetMaterial();
```

```
SurfaceTerrain
Error: undefined reference to `temp.SurfaceOBJ._init'
referenced from `pure nothrow @trusted temp.SurfaceOBJ core.lifetime._d_newclassT!(temp.SurfaceOBJ)._d_newclassT()
Error: undefined reference to `temp.SurfaceOBJ._Class'
referenced from `pure nothrow @trusted temp.SurfaceOBJ core.lifetime._d_newclassT!(temp.SurfaceOBJ)._d_newclassT() /scene.d'
Error: undefined reference to `temp.SurfaceOBJ._Class'
referenced from `pure nothrow @safe temp.SurfaceOBJ temp.ResourceManager.AddResource!(temp.SurfaceOBJ).AddResource
OBJ)`
Error: undefined reference to `pure @safe immutable(char)[] temp.Vec3!(float).Vec3.toString()'
referenced from `@safe void std.format.internal.write.formatObject!(std.stdio.File.LockingTextWriter, temp.Vec3!(f
har).formatObject(ref std.stdio.File.LockingTextWriter, ref temp.Vec3!(float).Vec3, scope ref const(std.format.spec.Forma
FormatSpec))
Error: undefined reference to `temp.Vec3!(float).Vec3._init'
referenced from `pure nothrow @nogc @safe temp.Vec3!(float).Vec3 temp.Vec3!(float).Vec3.opBinary!("+")opBinary(te
t).Vec3)`
Error: undefined reference to `temp.Application._init'
referenced from `pure nothrow @trusted temp.Application core.lifetime._d_newclassT!(temp.Application)._d_newclassT
Error: undefined reference to `temp.Application._Class'
referenced from `pure nothrow @trusted temp.Application core.lifetime._d_newclassT!(temp.Application)._d_newclassT
perhaps `.d' files need to be added on the command line, or use `-i' to compile imports
Error: linker exited with status 1
```

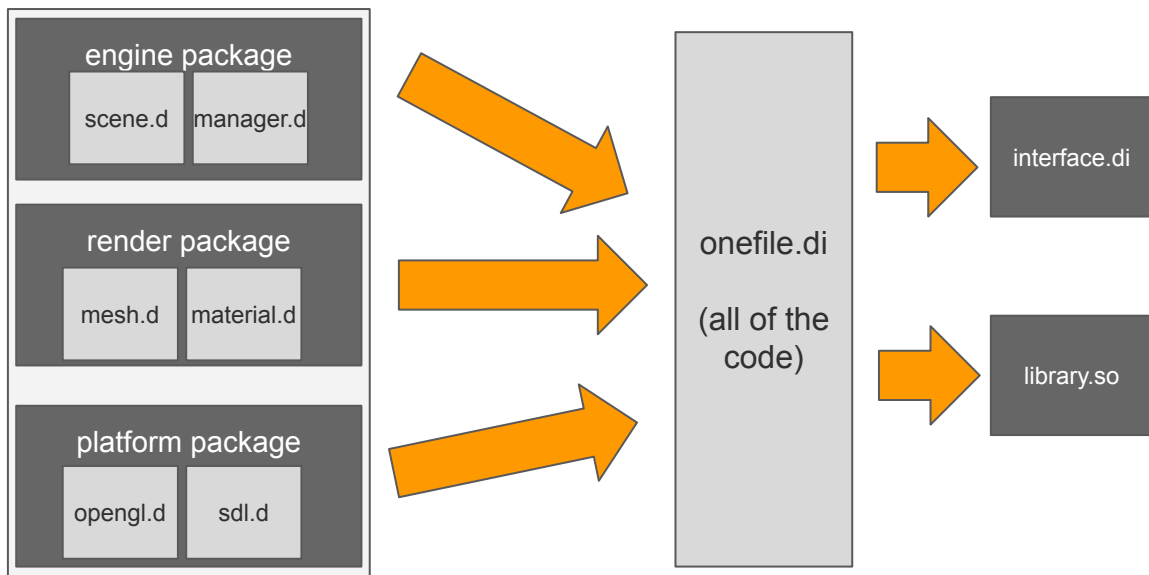
Game/Graphics engine -- one library, one .di file: Attempt # 3

- At some point, I thought about using 'nm' to lookup symbol names, and then rewrite function pointers for them -- that's probably pretty reasonable thing to do later on
 - It's really cool that you have 'core.demangle' right in D -- no need for external tools
 - But this still does not seem necessary at this point in the project.

```
1 import std.stdio;
2 import core.demangle;
3
4 //function _D11application11Application11RemoveSceneMFC5scene5SceneZv = _D11temp11Application11RemoveSceneMFC5scene5SceneZv;
5
6 void main(){
7     char[] buffer = demangle("_D11application11Application6__initZ");
8     writeln(buffer);
9
10 //printMessage();
11 }
```

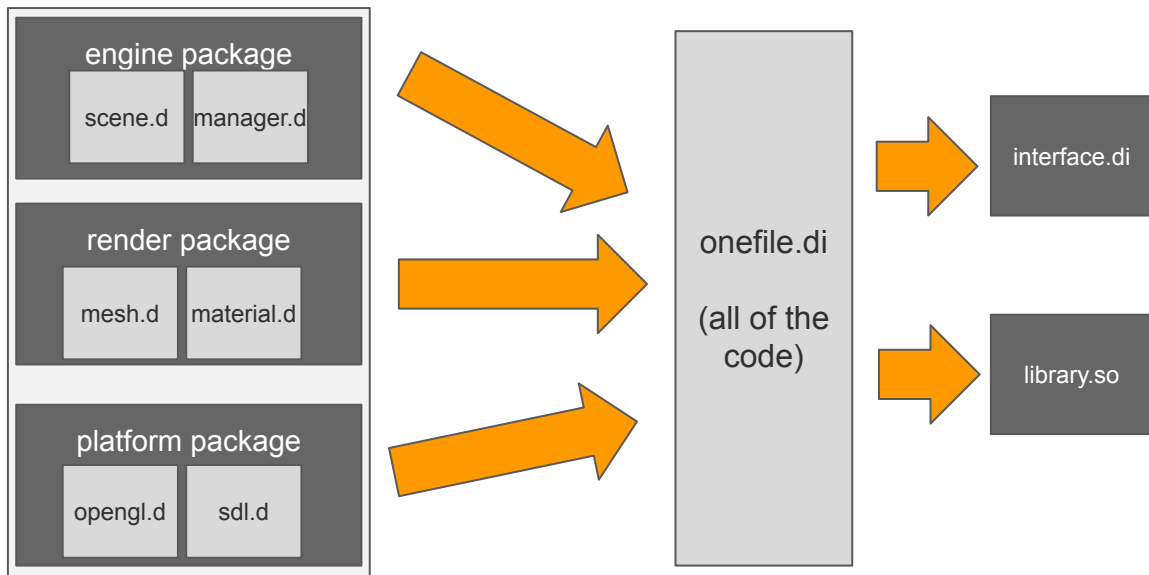
Game/Graphics engine -- one library, one .di file: **Solution!** (1/5)

- Just concatenate all of my .d files together into one giant .d file, and then generate the .di interface
 - Note: I remove the 'module name' of every .d file and end with '1 module'
 - Compilation is simple and fast
 - Can then generate the interface automatically with a flat structure
 - **Problem “mostly” solved**



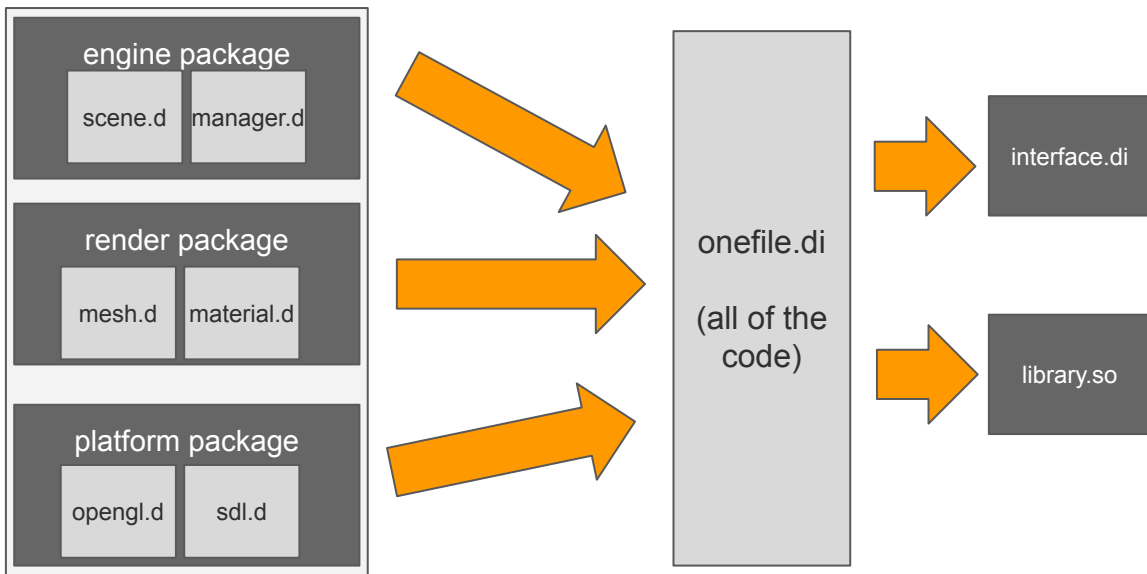
Game/Graphics engine -- one library, one .di file: **Solution!** (2/5)

- It's perhaps a little silly, but having 'onefile.di' with everything in it works!
 - It's easy to search and have all definitions in a file
 - My experience has been compiling one file is faster than linking lots of files



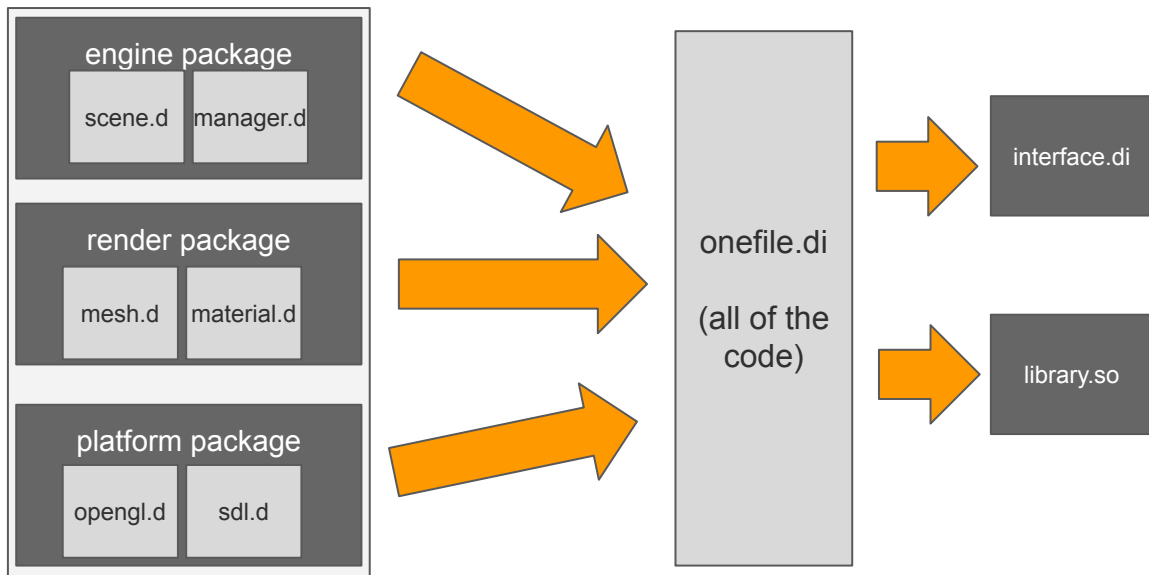
Game/Graphics engine -- one library, one .di file: **Solution!** (3/5)

- I kind of even like it for version control
 - Bad practice to have a 10,000+ line file?
 - It's fine for a 1-person project, my computer/version control works for me, not the other way around.



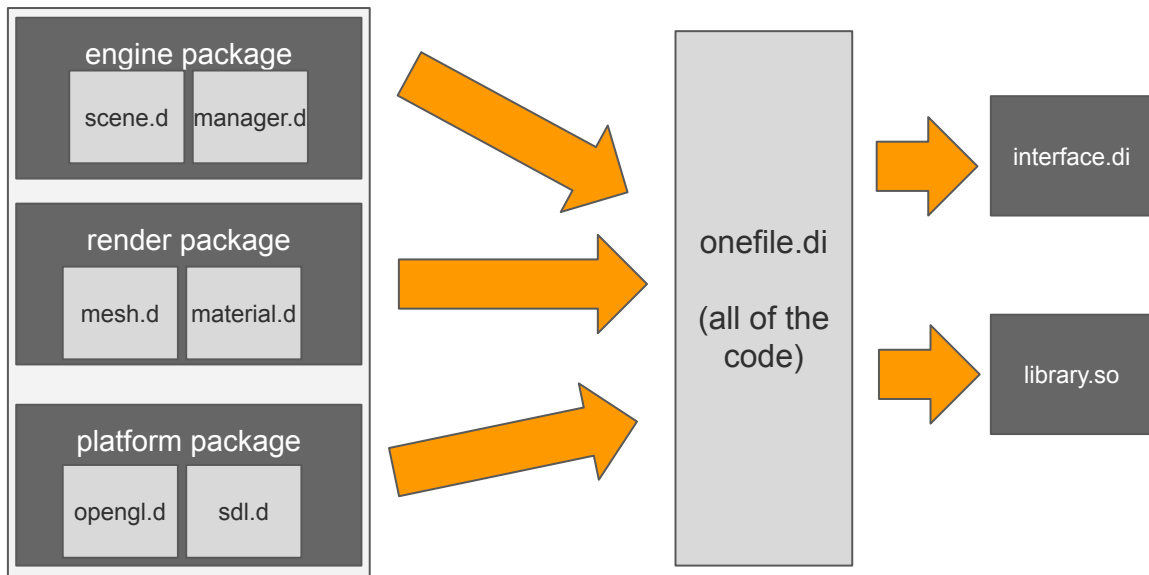
Game/Graphics engine -- one library, one .di file: **Solution!** (4/5)

- I kind of even like it for version control
 - Bad practice to have a 10,000+ line file?
 - It's fine for a 1-person project, my computer/version control works for me, not the other way around.
- ***Steam Achievement unlocked***
 - *Congratulations on starting your journey inventing yet another custom build system!*



Game/Graphics engine -- one library, one .di file: **Solution!** (5/5)

- I kind of even like it for version control
 - Bad practice to have a 10,000+ line file?
 - It's fine for a 1-person project, my computer/version control works for me, not the other way around.
- *Steam Achievement unlocked*
 - *Congratulations on starting your journey inventing yet another custom build system!*



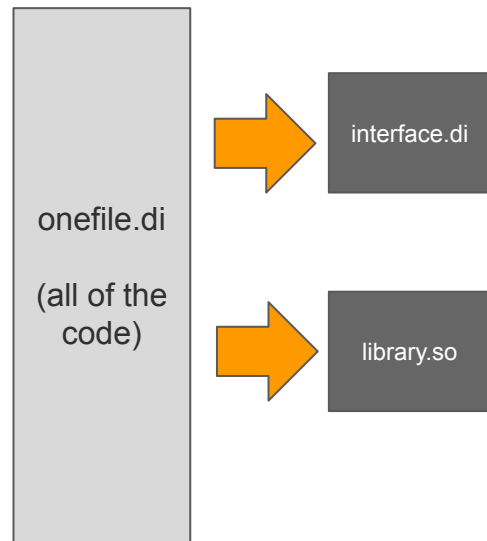
Greenspun's rule: Any sufficiently complicated *C* or *Fortran* program contains an *ad hoc*, informally-specified, bug-ridden, slow implementation of half of Common Lisp.

-- can be adapted to :

"Any sufficiently complicated codebase creates a new build system" (my paraphrase)

The Template Challenge (1/3)

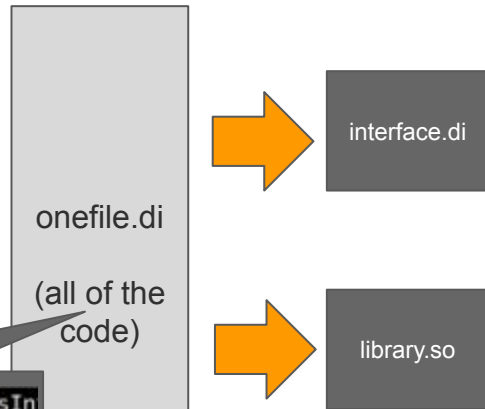
- Okay -- I have everything in **onefile.di**
- This provides a flat interface
 - No package hierarchy for users to worry about or symbols to demangle/version
 - Easier to build a 'C-interface' for portability
- So client should only need the [interface.di](#) file
-- no need to expose all details?



The Template Challenge (2/3)

- Okay -- I have everything in **onefile.d**
- This provides a flat interface
 - No package hierarchy for users to worry about or symbols to demangle/version
 - Easier to build a 'C-interface' for portability
- So client should only need the [interface.di](#) file
-- no need to expose all details?
 - Hmm, what about templates?

```
195 struct Vec3(T = float) if (__traits(isIn
196 {
197     this(T value)
198     {
199         this.x = value;
200         this.y = value;
201         this.z = value;
202     }
203     this(Vec2!T v, T z)
204     {
205         this.x = v.x;
206         this.y = v.y;
207         this.z = z;
208     }
209 }
```



The Template Challenge (3/3)

- Templates are great **But -- there is a question**
 - How do I distribute an **public facing interface to users without exposing code?**
 - The **top-right** is the .di interface with all details provided.
- Now, for a math library -- maybe not a huge detail, but what if I don't yet want to expose all of the engine source.
 - So how to deal with this?
 - (next slide)

```
246 /// Vector with 3-dimensions
247 struct Vec3(T=float)
248 { if(__traits(isIntegral,T) || __traits(isFloating,T))
249 {
250     /// Constructor for a vec3 type
251     /// Repeats every value
252     this(T value){
253         this.x = value;
254         this.y = value;
255         this.z = value;
256     }
257     /// Constructor for a vec3 type from a Vec2
258     this(Vec2!T v, T z){
259         this.x = v.x;
260         this.y = v.y;
261         this.z = z;
262     }
263 }
```

```
6 // Common alias for types
7 // Note: Most commonly we use 'float' type for vec2, so I omit the suffix.
8 // This is consistent with the matrix data types.
9 alias vec2 = Vec2!float;
10 alias vec2i = Vec2!int;
11 alias vec2d = Vec2!double;
12 // Alias 'Point2' for the 'Vec2' type.
13 alias Point2f = vec2;
14 alias Point2i = vec2i;
15 alias Point2d = vec2d;
```

The Template Challenge: Answer #1

- Answer 1: Don't worry about it
 - Templates are useful, and 'template constraints' are useful information to clients using types.
 - I potentially avoid many bugs using 'template constraints' on types (See top-right) or otherwise avoid duplicating code.
 - (bottom-right) I provide also lots of useful aliases to match types in glsl shader code.

```
246 /// Vector with 3-dimensions
247 struct Vec3(T=float)
248     if(__traits(isIntegral,T) || __traits(isFloating,T))
249 {
250     /// Constructor for a vec3 type
251     /// Repeats every value
252     this(T value){
253         this.x = value;
254         this.y = value;
255         this.z = value;
256     }
257     /// Constructor for a vec3 type from a Vec2
258     this(Vec2!T v, T z){
259         this.x = v.x;
260         this.y = v.y;
261         this.z = z;
262     }
263 }
```

```
6 // Common alias for types
7 // Note: Most commonly we use 'float' type for vec2, so I omit the suffix.
8 // This is consistent with the matrix data types.
9 alias vec2 = Vec2!float;
10 alias vec2i = Vec2!int;
11 alias vec2d = Vec2!double;
12 // Alias 'Point2' for the 'Vec2' type.
13 alias Point2f = vec2;
14 alias Point2i = vec2i;
15 alias Point2d = vec2d;
```

The Template Challenge: Answer #2

- Answer 2: Reach for metaprogramming
 - *I prototyped a metaprogramming system to generate instantiations of code that I need -- it kind of worked, but my solution seemed brittle.*
 - One solution might be to annotate functions with UDA

```
91 ///
92 void GenerateFreeFunction(alias s)(bool isMemberFunction){
93
94     mixin(`
95         // Write out function signatures
96         static if(!__traits(compiles,isFunction!(`~s~`))) {
97             return;
98         }
99
100         static if (isFunction!(`~s~`)){
101             // Return if it is not a public function
102             if(!IsPublic!(`~s~`)){
103                 return;
104             }
105             alias parameterNames = ParameterIdentifierTuple!(`~s~`);
106             alias parametertypes = Parameters!(`~s~`);
107             alias returnType = ReturnType!(`~s~`);
108             writeln("\t// Function");
109
110             string functionName = `~s.stringof~`;
111             // Trim the namespace if this is a member function
112             // TODO: May need to fix nested classes/structs
113             if(isMemberFunction){
```

The Template Challenge: Answer #3 (1/3)

- Answer 3:
 - For at least functions, it is easy to simply wrap templated instances into a non-templated function.

```
/// Finds all nodes of a specific type.
T[] FindAllNodes(T)(ISceneNode start){
    if(start is null){
        assert(false, "Error, start node is null for FindAllNodes");
        return null;
    }

    // Collect nodes of type 'T' into a result.
    T[] resultNodes;

    // Perform depth-first traversal
    ISceneNode current = mRootNode;
    ISceneNode[] stack = current.mChildren;
    while(stack.length > 0){
```

```
/// Helper function to return a dynamic array of all mesh nodes from scene tree.
/// This will retrieve the nodes in a depth-first traversal.
/// This function may be helpful for listing nodes, or otherwise searching or applying
/// operations to all nodes of a specific type or sub-type.
MeshNode[] GetAllMeshNodes(){
    return mSceneTree.FindAllNodes!MeshNode(mSceneTree.GetRootNode());
}

/// Helper function to return a dynamic array of all light nodes from scene tree.
/// This will retrieve the nodes in a depth-first traversal.
/// This function may be helpful for listing nodes, or otherwise searching or applying
/// operations to all nodes of a specific type or sub-type.
LightNode[] GetAllLightNodes(){
    return mSceneTree.FindAllNodes!LightNode(mSceneTree.GetRootNode());
}
```

The Template Challenge: Answer #3 (2/3)

- Ultimately -- I just need two files then if I want to expose generic types/functions to users --
 - one file for non-templated functions/data types that clients use most often
 - and a second file if I want to expose 'generics' that writers of game code can also use to instantiate their own templates
- Ultimately, I'm just thinking about how to give clients full access to the D language in a simple way.

```
/// Finds all nodes of a specific type.
T[] FindAllNodes(T)(ISceneNode start){
    if(start is null){
        assert(false, "Error, start node is null for FindAllNodes");
        return null;
    }

    // Collect nodes of type 'T' into a result.
    T[] resultNodes;

    // Perform depth-first traversal
    ISceneNode current = mRootNode;
    ISceneNode[] stack = current.mChildren;
    while(stack.length > 0){
```

```
/// Helper function to return a dynamic array of all mesh nodes from scene tree.
/// This will retrieve the nodes in a depth-first traversal.
/// This function may be helpful for listing nodes, or otherwise searching or applying
/// operations to all nodes of a specific type or sub-type.
MeshNode[] GetAllMeshNodes(){
    return mSceneTree.FindAllNodes!MeshNode(mSceneTree.GetRootNode());
}

/// Helper function to return a dynamic array of all light nodes from scene tree.
/// This will retrieve the nodes in a depth-first traversal.
/// This function may be helpful for listing nodes, or otherwise searching or applying
/// operations to all nodes of a specific type or sub-type.
LightNode[] GetAllLightNodes(){
    return mSceneTree.FindAllNodes!LightNode(mSceneTree.GetRootNode());
}
```

The Template Challenge: Answer #3 (3/3)

- Generic function (top code snippet) useful for me to write on the engine side.
- For the client, I can also simply expose specific instances for types (bottom-image) with reasonable defaults.

```
/// Finds all nodes of a specific type.
T[] FindAllNodes(T)(ISceneNode start){
    if(start is null){
        assert(false, "Error, start node is null for FindAllNodes");
        return null;
    }

    // Collect nodes of type 'T' into a result.
    T[] resultNodes;

    // Perform depth-first traversal
    ISceneNode current = mRootNode;
    ISceneNode[] stack = current.mChildren;
    while(stack.length > 0){
```

```
/// Helper function to return a dynamic array of all mesh nodes from scene tree.
/// This will retrieve the nodes in a depth-first traversal.
/// This function may be helpful for listing nodes, or otherwise searching or applying
/// operations to all nodes of a specific type or sub-type.
MeshNode[] GetAllMeshNodes(){
    return mSceneTree.FindAllNodes!MeshNode(mSceneTree.GetRootNode());
}

/// Helper function to return a dynamic array of all light nodes from scene tree.
/// This will retrieve the nodes in a depth-first traversal.
/// This function may be helpful for listing nodes, or otherwise searching or applying
/// operations to all nodes of a specific type or sub-type.
LightNode[] GetAllLightNodes(){
    return mSceneTree.FindAllNodes!LightNode(mSceneTree.GetRootNode());
}
```

More Examples of D Language Magic

‘static if’ (1/2)

- It's not uncommon for me to write code that might look like this on the first iteration.
 - I just want to get something working.
- But of course, we can do better using ‘templates’
 - (As just discussed)

```
Pipeline AddResource(Pipeline resource){  
    mPipelines[resource.mPipelineName] = resource;  
    return mPipelines[resource.mPipelineName];  
}  
IMaterial AddResource(IMaterial resource){  
    mMaterials[resource.mMaterialName] = resource;  
    return mMaterials[resource.mMaterialName];  
}
```

'static if' (2/2)

- I can use templates to make a common function or type.
 - 'static if' ensures that per instantiation of the function I call into an expected pathway with a consistent interface.
 - Types that are missing, return an assert message otherwise -- this is great for me!

```
Pipeline AddResource(Pipeline resource){  
    mPipelines[resource.mPipelineName] = resource;  
    return mPipelines[resource.mPipelineName];  
}  
IMaterial AddResource(IMaterial resource){  
    mMaterials[resource.mMaterialName] = resource;  
    return mMaterials[resource.mMaterialName];  
}
```



```
26  /// Add resources to the resource manager  
27  T AddResource(T)(T resource){  
28      static if(is(T : Pipeline)){  
29          mPipelines[resource.mPipelineName] = resource;  
30          return cast(sizeof(resource))mPipelines[resource.mPipelineName];  
31      }  
32      static if(is(T : IMaterial)){  
33          mMaterials[resource.mMaterialName] = resource;  
34          return cast(sizeof(resource))mMaterials[resource.mMaterialName];  
35      }  
    ...
```

Note: As with everything there is a trade-off however. If I later expose a C-API, I might actually prefer repeating some of these functions to have control over the exported function names.

Template Usage is Aggressive -- It's a great tool in Dlang (1/5)

- In my engine code I have lots of functions taking full advantage of templates.
 - (See top code snippet)
- However, I also provide in the engine 'flat C API' functions for most common operations I would expect a client to use in game code and can publicly expose
 - (See bottom snippet)
 - I've found this is simply easier to write for the common day-to-day tasks, and I want to control some of what I expose in the engine.

```
/// Finds all nodes of a specific type 'T'.  
/// Typically this is preferred versus doing many calls of 'FindNode'  
T[] FindAllNodesOfType(T)(){  
    if(this is null){  
        assert(false, "Error, start node is null for FindAllNodes");  
        return null;  
    }  
  
    // Collect nodes of type 'T' into a result.  
    T[] resultNodes;  
  
    // Perform depth-first traversal  
    ISceneNode current = mRootNode;  
    ISceneNode[] stack = current.mChildren;  
    while(stack.length > 0){  
        /// End of array (top of stack) is now current node.  
        current = stack[$-1];  
        if(typeid(current) == typeid(T)){  
            resultNodes ~= cast(T)current;  
        }  
        // Remove the current node from stack  
        stack.popBack();  
        // Append all of the children to the back of the stack  
        stack ~= current.mChildren;  
    }  
  
    return resultNodes;  
}
```

```
LightNode[] GetAllLightNodes(){  
    return mSceneTree.FindAllNodesOfType!LightNode();  
}
```

Template Usage is Aggressive -- It's a great tool in Dlang (2/5)

- Can even go further and make things more generic with a predicate function
- Now I can get rid of what is on the left side

```
/// Finds all nodes of a specific type 'T'.
/// Typically this is preferred versus doing many calls of 'FindNode'
T[] FindAllNodesOfType(T)(){
    if(this is null){
        assert(false, "Error, start node is null for FindAllNodes");
        return null;
    }

    // Collect nodes of type 'T' into a result.
    T[] resultNodes;

    // Perform depth-first traversal
    ISceneNode current = mRootNode;
    ISceneNode[] stack = current.mChildren;
    while(stack.length > 0){
        // End of array (top of stack) is now current node.
        current = stack[$-1];
        if(typeid(current) == typeid(T)){
            resultNodes ~= cast(T)current;
        }
        // Remove the current node from stack
        stack.popBack();
        // Append all of the children to the back of the stack
        stack ~= current.mChildren;
    }

    return resultNodes;
}
```

```
104 /// Finds all nodes that satisfy a predicate..
105 /// Typically this is preferred versus doing many calls of 'FindNode'
106 ISceneNode[] FindAllNodes(alias Predicate)(SceneTree tree) {
107     if(tree is null){
108         assert(false, "Error, tree is null for FindAllNodes");
109         return null;
110     }
111
112     // Collect nodes of type into a result.
113     ISceneNode[] resultNodes;
114
115     // Perform depth-first traversal
116     ISceneNode current = tree.GetRootNode();
117     ISceneNode[] stack = current.mChildren;
118     while(stack.length > 0){
119         // End of array (top of stack) is now current node.
120         current = stack[$-1];
121         if(Predicate(current)){
122             resultNodes ~= current;
123         }
124         // Remove the current node from stack
125         stack.popBack();
126         // Append all of the children to the back of the stack
127         stack ~= current.mChildren;
128     }
129
130     return resultNodes;
131 }
```

Template Usage is Aggressive -- It's a great tool in Dlang (3/5)

- Here's examples of new functions, the bottom using the 'most generic' version

```
LightNode[] GetAllLightNodes(){  
    return mSceneTree.FindAllNodesOfType!LightNode();  
}
```

a predicate...
versus doing many calls of 'FindNode'
Predicate)(SceneTree tree) {
 ...
 is null for FindAllNodes");

```
110 }  
111  
112 // Collect nodes of type into a result.  
113 ISceneNode[] resultNodes;  
114
```

```
LightNode[] GetAllLightNodes(){  
    LightNode[] nodes = cast(LightNode[])(mSceneTree.FindAllNodes!(a=> typeid(a)==typeid(LightNode)));  
    return nodes;  
}
```

While the top version is cleaner, the bottom version makes use of a more generic templated function, and opens up the ability to write all sorts of querying functions much more easily thanks to templates.

```
129  
130 return resultNodes;  
131 }
```

Template Usage is Aggressive -- It's a great tool in Dlang (4/5)

- Here's just another example in the 'wild' in my code to demonstrate the benefit.
- Now I can find nodes by 'name' -- again just one single templated function.

```
104 /// Finds all nodes that satisfy a predicate..  
105 /// Typically this is preferred versus doing many calls of 'FindNode'  
106 ISceneNode[] FindAllNodes(alias Predicate)(SceneTree tree) {  
107     if(tree is null){
```

```
ISceneNode[] nodeList = GetSceneTree().FindAllNodes!(a=> a.mNodeName.indexOf("boundingbox") > 0)();
```

```
111  
112     // Collect nodes of type into a result.  
113     ISceneNode[] resultNodes;  
114  
115     // Perform depth-first traversal  
116     ISceneNode current = tree.GetRootNode();  
117     ISceneNode[] stack = current.mChildren;  
118     while(stack.length > 0){  
119         /// End of array (top of stack) is now current node.  
120         current = stack[$-1];  
121         if(Predicate(current)){  
122             resultNodes ~= current;  
123         }  
124         // Remove the current node from stack  
125         stack.popBack();  
126         // Append all of the children to the back of the stack  
127         stack ~= current.mChildren;  
128     }  
129  
130     return resultNodes;  
131 }
```

Note: Observe the function signature changed slightly -- however, thanks to UFCS -- no code actually changes where 'FindAllNodes' is called.

The reason for the change was because of the 'dual context' needed for the predicate function and object (See: <https://bugzilla-archive.dlang.org/bugs/5710/>) or

<https://forum.dlang.org/thread/riockmzbpbbsvylajolyl@forum.dlang.org#post-riockmzbpbbsvylajolyl:40forum.dlang.org>

ing (5/5)

e function

```
/// Finds all nodes of a specific type 'T'.
/// Typically this is preferred versus doing many calls of 'FindNode'
T[] FindAllNodesOfType(T){
    if(this is null){
        assert(false, "Error, start node is null for FindAllNodes");
        return null;
    }

    // Collect nodes of type 'T' into a result.
    T[] resultNodes;

    // Perform depth-first traversal
    ISceneNode current = mRootNode;
    ISceneNode[] stack = current.mChildren;
    while(stack.length > 0){
        // End of array (top of stack) is now current node.
        current = stack[$-1];
        if(typeid(current) == typeid(T)){
            resultNodes ~= cast(T)current;
        }
        // Remove the current node from stack
        stack.popBack();
        // Append all of the children to the back of the stack
        stack ~= current.mChildren;
    }

    return resultNodes;
}
```

```
104 /// Finds all nodes that satisfy a predicate..
105 /// Typically this is preferred versus doing many calls of 'FindNode'
106 ISceneNode[] FindAllNodes(alias Predicate)(SceneTree tree) {
107     if(tree is null){
108         assert(false, "Error, tree is null for FindAllNodes");
109         return null;
110     }
111
112     // Collect nodes of type into a result.
113     ISceneNode[] resultNodes;
114
115     // Perform depth-first traversal
116     ISceneNode current = tree.GetRootNode();
117     ISceneNode[] stack = current.mChildren;
118     while(stack.length > 0){
119         // End of array (top of stack) is now current node.
120         current = stack[$-1];
121         if(Predicate(current)){
122             resultNodes ~= current;
123         }
124         // Remove the current node from stack
125         stack.popBack();
126         // Append all of the children to the back of the stack
127         stack ~= current.mChildren;
128     }
129
130     return resultNodes;
131 }
```

Deprecated

- Here's an example where I am encouraging 'myself' to use the explicit (versus the templated) form of a function.
 - deprecated in D I have found very useful (along with UDA's)
 - As you've noticed -- it takes time to find the right abstraction level, and getting a list of deprecations at compile-time is helpful!

```
99  /// Retrieves a node from the scene tree.
100  /// This function will attempt to cast the node to the type
101  /// NOTE: This function can be left in, however, the 'GetMeshNode' and 'GetLightNode' are useful because
102  ///      they could also be used with a C-ABI, and avoid templates.
103  deprecated("Prefer GetMeshNode or GetLightNode") T GetNode(T)(string name){
104      return cast(T)mSceneTree.FindNode(name);
105  }
106
107  /// Retrieves a Mesh Node from the scene tree based on its name
108  MeshNode GetMeshNode(string name)
109  {
110      ISceneNode node = mSceneTree.FindNode(name);
111      if(node != null){
112          assert(typeid(node)==typeid(MeshNode),"Found node, but it is not a MeshNode, perhaps it is LightNode, GroupNode, or something else?");
113      }
114      return cast(MeshNode)node;
115  }
116
117  /// Retrieves a Light Node from the scene tree based on its name
118  LightNode GetLightNode(string name)
119  {
120      ISceneNode node = mSceneTree.FindNode(name);
121      if(node != null){
122          assert(typeid(node)==typeid(LightNode),"Found node, but it is not a LightNode, perhaps it is MeshNode, GroupNode or something else?");
123      }
124      return cast(LightNode)node;
125  }
```

- The 'deprecated' functionality here still works, I use it as a 'training tool' for finding the cleanest way to use an API
 - I actually don't want to remove the function necessarily -- in fact, I might have a UDA and use a mixin to otherwise report 'preferred usage'
 - However, the 'templated' version still allows me as the developer to 'rapidly prototype' and add new generic types.

Phobos and UFCS make it easy to add features

- Here I took a depth-first traversal of my scene tree, and made it easy to filter away nodes that are not active on the same line.

```
// Append all of the children to the back of the stack
if(current.mChildren.length > 0){
  stack ~= current.mChildren;
}
```



```
// Append all of the children to the back of the stack
if(current.mChildren.length > 0){
  stack ~= current.mChildren.filter!(node=>node.mIsActive = true).array;
}
```

UFCS Makes Code Beautiful (and easier to understand)

- UFCS such a great feature.
 - Can always use the builder pattern
 - But allows me to ‘free my functions’ and not restrict them to member functions -- often times also allowing me to build the ‘flat C-API’ and remove OO code.
 - I might also notice that ‘GetRenderer().GetRenderState()’ can just be a function called ‘GetCurrentRenderState’

```
GetRenderer().GetRenderState().SetSceneTreeFilterFunction(null);  
GetRenderer().GetRenderState().SetBlending(true);  
// Enable the clipping plane  
GetRenderer().GetRenderState().SetClipDistance0(true);  
GetRenderer().GetRenderState().SetClippingPlane(vec4(0,1,0,-GetMeshNode("water").GetWorldPosition.y));
```



```
GetRenderer().GetRenderState().SetSceneTreeFilterFunction(null)  
    .SetBlending(true)  
    .SetClipDistance0(true)  
    .SetClippingPlane(vec4(0,1,0,-GetMeshNode("water").GetWorldPosition.y));
```

Other Engine Notes

Minimizing Engine Dependencies

- Here are the dependencies for the engine
 - bindbc.sdl
 - bindbc.opengl
 - Phobos 2
 - imgui
 - and dear_bindings to generate a 'c' interface for imgui.
- I suppose I have a severe case of 'not invented here' ([wiki](#)) when it comes to dependencies -- but I want to keep things as simple minimal as possible.
 - Philosophy is to only expand as platform support is needed.

D Binding Notes (1/2)

- I did not want to use a lot of dependencies, but at some point one must succumb to some.
- Luckily D has a lot useful ways to interface with both C and C++ libraries (and Objective-C)
- This can include ‘mangling’ the names right away
 - Often I would simply just use ‘nm’ to look at the symbols in the binary, and then grab a symbol or two I needed.
 - I could then make the function call as such:

```
struct SDL_Window;  
struct SDL_GL_Context;  
pragma(mangle, "_Z28ImGui_ImplSDL3_InitForOpenGL10SDL_WindowPv")  
extern(C++) bool ImGui_ImplSDL3_InitForOpenGL(SDL_Window* window, void* sdl_gl_context);
```

D Binding Notes (2/2)

- bitfields
 - bitfields are great, but don't use them in your library if you're exposing them in a public api.
 - Occasionally some interesting translations when found in libraries -- easiest fix I could think of was to do something like this to match the layout.

```
struct ImFontGlyph_t
{
    uint32_t Colored : 1;
    uint32_t Visible : 1;
    uint32_t SourceIdx : 4;
    uint32_t Codepoint : 26;
    float AdvanceX = void;
    float X0 = void;
    float Y0 = void;
    float X1 = void;
    float Y1 = void;
    float U0 = void;
```

```
struct ImFontGlyph_t
{
    uint32_t bitfield; // uint32_t Colored : 1;
    // uint32_t Visible : 1;
    // uint32_t SourceIdx : 4;
    // uint32_t Codepoint : 26;
    float AdvanceX = void;
    float X0 = void;
    float Y0 = void;
    float X1 = void;
    float Y1 = void;
    float U0 = void;
```

```
Error: cannot take address of bit-field `Colored`
Error: cannot take address of bit-field `Visible`
Error: cannot take address of bit-field `SourceIdx`
Error: cannot take address of bit-field `Codepoint`
```

(For more see)

- While this was fresh in my mind, I did a little livestream in July on the first iteration.
 - https://youtube.com/live/BQmwbHw_BFo
 - (I've since cleaned up a few things like the pragmas



D Language Advantage

- Code compiles fast!
 - No need for hot swapping for code iteration -- yet
 - Note: I will eventually need it once I start loading more resources however -- useful to avoid time for disk access of assets
- I have somewhat trivially increased asset file loads using async file I/O with Phobos.
 - See D tutorial: <https://www.youtube.com/watch?v=Kor8HwIkXbw>
- As promised from the abstract my setup
 - **(See next slide)**

Engine dub configuration

- My dub configuration for the engine code looks something like this.
- I run a few commands to generate ctags, run scanner, and otherwise generate the 'onefile' interface.

```
1 {
2     "authors": [
3         "mike"
4     ],
5     "targetType": "dynamicLibrary",
6     "copyright": "Copyright © 2026, mike",
7     "dependencies": {
8         "bindbc-opengl": "~>1.1.1",
9         "bindbc-sdl": "~>2.2.0"
10    },
11    "versions": [
12        "GL_46"
13    ],
14    "preGenerateCommands": ["echo ctags-universal `find ./source2 -type f -name \"*.d\"`", "dub run dscanner -- --ctags source2/**/*.d > tags"],
15    "dflags": ["-mixin=mixin.txt", "-Hd=public_interface", "-H", "-Hf=./public_interface/onefile.d"],
16    "description": "Mike's Game and Graphics Engine",
17    "license": "proprietary",
18    "name": "engine"
19 }
```

Setup - Screen 1

- Left window is constantly compiling
- top-right window is my todo list (.plan file)
- bottom-right I use for grep and searching code

```
jam.d Adding file: ./source2/render/uniform.d
jam.d Adding file: ./source2/render/light/lightnode.d
jam.d Adding file: ./source2/render/buffer.d
jam.d Adding file: ./source2/render/renderer.d
jam.d Adding file: ./source2/platform/openssl abstraction.d
jam.d Adding file: ./source2/platform/gltf.d
jam.d Adding file: ./source2/platform/public interface.d
jam.d Adding file: ./source2/platform/logger.d
jam.d Adding file: ./source2/platform/sdl abstraction.d
jam.d Adding file: ./source2/engine/scenes.d
jam.d Adding file: ./source2/engine/application.d
jam.d Adding file: ./source2/engine/scenetree.d
jam.d Adding file: ./source2/engine/graphics window.d
jam.d Adding file: ./source2/engine/framerate.d
jam.d Adding file: ./source2/engine/scenenode.d
jam.d Adding file: ./source2/engine/camera.d
jam.d Adding file: ./source2/engine/util.d
jam.d Adding file: ./source2/engine/resourcemanager.d
jam.d Adding file: ./source2/linear/mat.d
jam.d Adding file: ./source2/linear/test/test.d
jam.d Adding file: ./source2/linear/vec.d
jam.d Adding file: ./source2/linear/vec.d
Pre-gen Running commands for engine
tags universal ./source2/render/image.d ./source2/render/geometry/vertexformats.d ./source2/render
geometry/package.d ./source2/render/geometry/surface.d ./source2/render/geometry/collision.d ./sou
ce2/render/geometry/object.d ./source2/render/geometry/terrain.d ./source2/render/text
/package.d ./source2/render/text/text2d.d ./source2/render/package.d ./source2/render/pipeline.d
./source2/render/renderer/target.d ./source2/render/textures.d ./source2/render/mesh.d ./source2/render/
graphicserror.d ./source2/render/material.d ./source2/render/uniform.d ./source2/render/light/light
node.d ./source2/render/light/package.d ./source2/render/buffer.d ./source2/render/renderer.d ./sou
ce2/platform/openssl abstraction.d ./source2/platform/package.d ./source2/platform/gltf.d ./sourc
e2/platform/public interface.d ./source2/platform/logger.d ./source2/platform/sdl abstraction.d /s
ource2/engine/package.d ./source2/engine/scenes.d ./source2/engine/application.d ./source2/engine/sc
enotree.d ./source2/engine/graphics window.d ./source2/engine/framerate.d ./source2/engine/scenenod
e.d ./source2/engine/camera.d ./source2/engine/util.d ./source2/engine/resourcemanager.d ./source2/
platform/public interface.d ./source2/linear/package.d ./source2/linear/mat.d ./source2/linear/vec.d
test/test.d ./source2/linear/vec.d
Warning Warning: License in sub-package dcd:dsymbol is different than its parent package, this
is discouraged.
Starting Performing "debug" build using ./usr/bin/dmd for x86_64.
Up-to-date bindc-loader 1.1.5: target for configuration [noBC] is up to date.
Up-to-date bindc-opengl 1.1.1: target for configuration [dynamic] is up to date.
Up-to-date bindc-common 1.0.5: target for configuration [noBC] is up to date.
Up-to-date bindc-sdl 2.2.0: target for configuration [dynamic] is up to date.
Building engine -master: building configuration [library]
Linking engine
Finished To force a rebuild of up-to-date targets, run again with --force
ash returned exit code 0

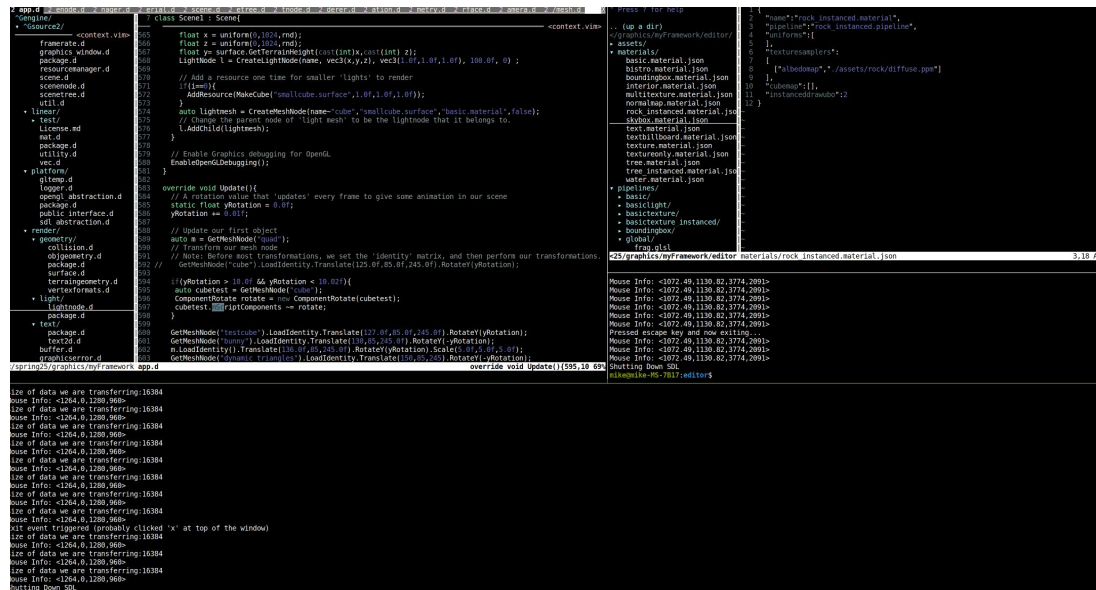
21 Rendering <context.vim>
28 - [ ] Need to add to materials lighting information.
29 - [ ] Need to add in view frustum culling.
30 - [ ] Need to upgrade to update to OpenGL 4.5/4.6 for all materials and separate
out renderers potentially for a 'compatibility mode' of some sort.
31 - [ ] Implement a depth-prepass and measure its impact
32 - [ ] Most likely occlusion queries will help validate if this is working, a
d I'll need to do frustum culling prior
33 - [ ] https://wikis.khronos.org/OpenGL/EarlyFragmentTest
34 - [x] Fix terrain to use primitive restart and issue fewer draw calls.
35 - [x] obj files can now be exported.
36 - [ ] Need to add a way to output the 'baked' binary data for faster loads.
37 - [ ] Uniform Buffer Objects for instancing currently are special cased and have
duplicate shaders, I started by adding something in the .json for materials to indic
te which buffer to draw into, but need to figure out how to translate this into avoi
ing duplicating each pipeline per material.
38 - [ ] 'skybox' currently has a very 'special case' in the rendering traversal fo
changing the state -- so need to manage that.
39 - [ ] Missing shadows
40 - Camera
41 - [ ] Need to have a camera 'lookToward' function or otherwise fix 'lookAt'. The
purpose is to then write some functions to 'lock' a camera onto a node.
42 - [ ] Transition still could be a bit smoother, especially on the first click. A
so does not work with trackpad properly.
43 - Lighting
44 - [ ] Tune the point lights 39,25 31%

mike@mike-HS-7B17:engine$ vim source/app.d
mike@mike-HS-7B17:engine$ vim source/app.d
mike@mike-HS-7B17:engine$
```

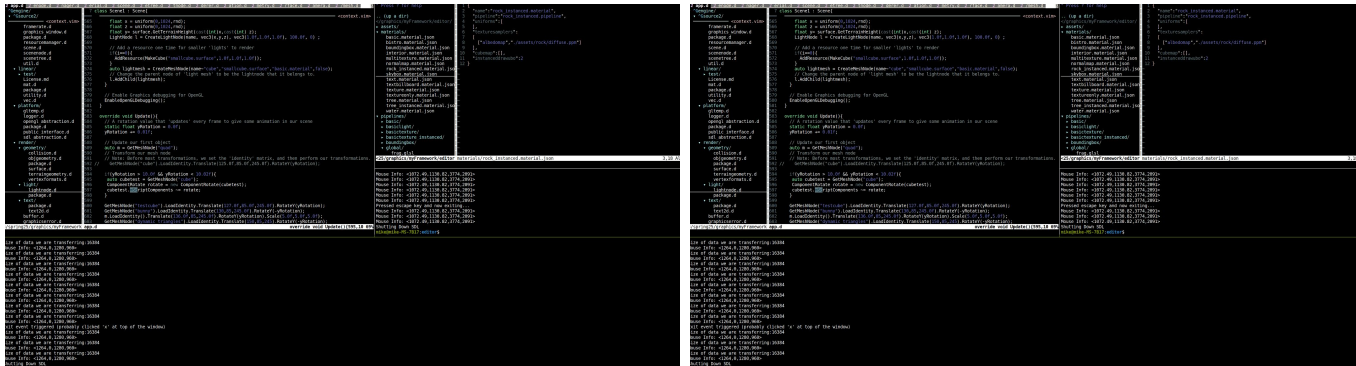
find . -name "*.d" | entr -s 'sh [build.sh](#)'
I use this command to constantly rebuild engine ([build.sh](#))
essentially does 'dub build' on every change.

Setup - Screen 2

- top-left
 - I write d source in Vim
- top-right
 - Write glsl shader and material file code
- bottom-window
- I run another shell script to compile & run game.
 - This links engine shared library and current 'app.d' file into one program.



More information on my setup



- It's very simple -- I essentially just use Vim and constantly recompile, relink, and rebuild engine (< 2 seconds) every change.
- I use gdb to debug
 - Note: `pipe info variables | grep 'var'` is helpful if you have trouble in gdb finding a D symbol behind a module for autocomplete.
- I use renderdoc (free) to debug GPU/Graphics issues.

Graphics Engine Feature Dump

Graphics Engine

- On the graphics engine side of things you are doing a lot of your work writing shader code.
- Note:
 - My previous DConf talks talk about automating handling of things like 'attributes' in the vertex shader.

```
1 layout(location=0) in vec3 aPosition;
2 layout(location=1) in vec2 aTexCoords;
3
4 out vec2 vTexCoords;
5 out vec4 vWorldCoords;
6
7 uniform mat4 uModel;
8
9 void main()
10 {
11     vTexCoords = aTexCoords;
12     vWorldCoords = uModel * vec4(aPosition,1.0f);
13
14     // Clip geometry based on the clipping plane
15     vec4 worldPosition = uModel * vec4(aPosition,1.0f);
16     gl_ClipDistance[0] = dot(worldPosition, uClippingPlane0);
17
18     vec4 finalPosition = uProjection * uView * uModel * vec4(aPosition,1.0f);
19
20     // Note: Something subtle, but we need to use the finalPosition.w to do the perspective divide
21     gl_Position = vec4(finalPosition.x, finalPosition.y, finalPosition.z, finalPosition.w);
22 }
```

Graphics Engine - Automate Uniforms

- Handling uniforms and uniform buffers objects can be a bit of a nuisance.
- I effectively automated away all of that by wrapping uniform variables in a type.

```
52 class Uniform{
53     GLint mPipelineId;
54
55     // When a uniform is added to a material automatically.
56     GLint mCachedUniformLocation; // We cache
57
58     // Stores information about the uniform or
59     // This includes the uniform name, if it's
60     UniformTypeInfo mUniformTypeInfo;
61
62     void* mData = null;
63
64     /// Construct a uniform and encode type info
65     this(UniformTypeInfo u){
66         mUniformTypeInfo = u;
67     }
68 }
```

Graphics Engine - Automate Uniforms

- Uniforms can be 'set' and if they are a non-primitive type (e.g. a struct), then they are handled and transferred one field at a time.

```
/// Template to set a scalar type, built-in type, or a struct
/// Note:
/// CPU struct needs to match glsl struct layout.
void Set(T)(auto ref T value)
    //if(__traits(isPOD,T))
{
    lines: import std.traits;-----
    T* heapValue = new T;

    // For scalar types run this branch, otherwise convert the 'pod' type
    static if(__traits(isScalar,T)){
        // Copy value into our heap storage
        *heapValue = value;
        // Point our data to the heap allocated memory
        mData = cast(void*)heapValue;
    }
    else{
        lines: Debug and print out the type we are attempting to set in memory-----
        foreach(field ; FieldNameTuple!T){
            mixin("heapValue."~field.to!string~"=value."~field.to!string~";");
        }
        // Point our data to the heap allocated memory
        mData = cast(void*)heapValue;
    }
}
```

Graphics Engine - Automate Uniforms

- Inspired by D's 'TypeInfo' -- I end up creating a struct for all of the uniforms and uniform blocks when my glsl shaders are compiled.
 - Then per uniform, or per uniform buffer block, I can automate per frame the transfer of uniforms based on the objects material.

```
57 /// Introspection will build a 'map' of all the of uniforms found, and also add any Uniform Buffer Blocks
58 /// to the static pipeline map automatically.
59 UniformTypeInfo[string] Introspection(string pipelineName){
60     UniformTypeInfo[string] uniformMap;           // Names of the uniform variables used in the shader
61
62
63     GLuint programID = Pipeline.sPipelineProgramIDs[pipelineName];
64     writeln("=====");
65     writeln("Introspection on :", pipelineName, " (program ID = ",programID,")");
66     writeln("=====");
67
68     // Figure out how many uniforms and uniform blocks we have
69     GLint programUniformCount;
70     GLint programUniformBlocksCount;
71     glGetProgramiv(programID, GL_ACTIVE_UNIFORMS, &programUniformCount);
72     glGetProgramiv(programID, GL_ACTIVE_UNIFORM_BLOCKS, &programUniformBlocksCount);
73     writeln("Total Uniforms: ", programUniformCount);
74     writeln("Total Uniform blocks: ", programUniformBlocksCount);
75 }
```

Graphics Engine - Automate Uniforms

- So generally speaking, users of the engine just write shader code, and occasionally a material file if some 'state' is needed to be configured.
 - (Material files will later be automatically generated from GUI).
- Putting things into other storage types (e.g. UBO or SSBO) is thus trivial.
 - D and glsl introspection saves the day!

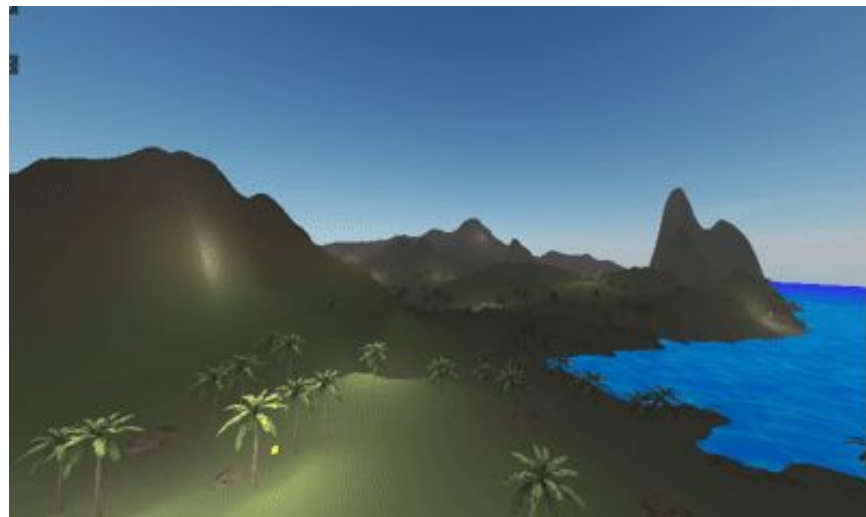
```
mike@mike-Gazelle:~/Dropbox/academi
materials/
├── basic.material.json
├── bistro.material.json
├── boundingbox.material.json
├── interior.material.json
├── multitexture.material.json
├── normalmap.material.json
├── rock_instanced.material.json
├── skybox.material.json
├── textbillboard.material.json
├── text.material.json
├── texture.material.json
├── textureonly.material.json
├── tree_instanced.material.json
├── tree.material.json
└── water.material.json

1 directory, 15 files
mike@mike-Gazelle:~/Dropbox/academi
pipelines/
├── basic
│   ├── frag.glsl
│   └── vert.glsl
├── basiclight
│   ├── frag.glsl
│   └── vert.glsl
├── basictexture
│   ├── frag.glsl
│   └── vert.glsl
├── basictexture_instanced
│   ├── frag.glsl
│   └── vert.glsl
├── boundingbox
│   ├── frag.glsl
│   └── vert.glsl
```

Summary

Summary

- Today I showed a small 3D Graphics/Game Engine I've handmade in D
- D has been awesome to quickly iterate and build out an engine
- I've talked about D being awesome for graphics programming previously (links) -- and my opinion has not changed.



Some previous talks on graphics:

- [DConf '24 | The Case for Graphics Programming in D Part 2 | Mike Shah](#)
- [The Case for Graphics Programming Using the D Language - Mike Shah - ACCU 2025](#)
- [More...](#)

Thank you DConf 2026!



3D Graphics Engine in D

with Mike Shah

Web: mshah.io
YouTube: www.youtube.com/c/MikeShah
Social: mikesah.bsky.social
Courses: courses.mshah.io
Talks: <http://tinyurl.com/mike-talks>

35 minutes | Audience: All
2:45pm - 3:20pm Fri, Sept 4, 2026

DConf